
Generative Adversarial Imitation Learning

Jonathan Ho
Stanford University
hoj@cs.stanford.edu

Stefano Ermon
Stanford University
ermon@cs.stanford.edu

Abstract

Consider learning a policy from example expert behavior, without interaction with the expert or access to reinforcement signal. One approach is to recover the expert’s cost function with inverse reinforcement learning, then extract a policy from that cost function with reinforcement learning. This approach is indirect and can be slow. We propose a new general framework for directly extracting a policy from data, as if it were obtained by reinforcement learning following inverse reinforcement learning. We show that a certain instantiation of our framework draws an analogy between imitation learning and generative adversarial networks, from which we derive a model-free imitation learning algorithm that obtains significant performance gains over existing model-free methods in imitating complex behaviors in large, high-dimensional environments.

1 Introduction

We are interested in a specific setting of imitation learning—the problem of learning to perform a task from expert demonstrations—in which the learner is given only samples of trajectories from the expert, is not allowed to query the expert for more data while training, and is not provided reinforcement signal of any kind. There are two main approaches suitable for this setting: behavioral cloning [20], which learns a policy as a supervised learning problem over state-action pairs from expert trajectories; and inverse reinforcement learning [25][18], which finds a cost function under which the expert is uniquely optimal.

Behavioral cloning, while appealingly simple, only tends to succeed with large amounts of data, due to compounding error caused by covariate shift [23][24]. Inverse reinforcement learning (IRL), on the other hand, learns a cost function that prioritizes entire trajectories over others, so compounding error, a problem for methods that fit single-timestep decisions, is not an issue. Accordingly, IRL has succeeded in a wide range of problems, from predicting behaviors of taxi drivers [31] to planning footsteps for quadruped robots [22].

Unfortunately, many IRL algorithms are extremely expensive to run, requiring reinforcement learning in an inner loop. Scaling IRL methods to large environments has thus been the focus of much recent work [7][14]. Fundamentally, however, IRL learns a cost function, which explains expert behavior but does not directly tell the learner how to act. Given that learner’s true goal often is to take actions imitating the expert—indeed, many IRL algorithms are evaluated on the quality of the optimal actions of the costs they learn—why, then, must we learn a cost function, if doing so possibly incurs significant computational expense yet fails to directly yield actions?

We desire an algorithm that tells us explicitly how to act by directly learning a policy. To develop such an algorithm, we begin in Section 3 where we characterize the policy given by running reinforcement learning on a cost function learned by maximum causal entropy IRL [31][32]. Our characterization introduces a framework for directly learning policies from data, bypassing any intermediate IRL step.

Then, we instantiate our framework in Sections 4 and 5 with a new model-free imitation learning algorithm. We show that our resulting algorithm is intimately connected to generative adversarial

networks [9], a technique from the deep learning community that has led to recent successes in modeling distributions of natural images: our algorithm harnesses generative adversarial training to fit distributions of states and actions defining expert behavior. We test our algorithm in Section 6 where we find that it outperforms competing methods by a wide margin in training policies for complex, high-dimensional physics-based control tasks over various amounts of expert data.

2 Background

Preliminaries $\overline{\mathbb{R}}$ will denote the extended real numbers $\mathbb{R} \cup \{\infty\}$. Section 3 will work with finite state and action spaces $\mathcal{S}$ and $\mathcal{A}$ to avoid technical machinery out of the scope of this paper (concerning compactness of certain sets of functions), but our algorithms and experiments later in the paper will run in high-dimensional continuous environments. Π is the set of all stationary stochastic policies that take actions in $\mathcal{A}$ given states in $\mathcal{S}$; successor states are drawn from the dynamics model $P(s'|s, a)$. We work in the γ -discounted infinite horizon setting, and we will use an expectation with respect a policy $\pi \in \Pi$ to denote an expectation with respect to the trajectory it generates: $\mathbb{E}_\pi[c(s, a)] \triangleq \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t c(s_t, a_t)]$, where $s_0 \sim p_0$, $a_t \sim \pi(\cdot|s_t)$, and $s_{t+1} \sim P(\cdot|s_t, a_t)$ for $t \geq 0$. We will use $\hat{\mathbb{E}}_\tau$ to denote empirical expectation with respect to trajectory samples τ , and we will always refer to the expert policy as π_E .

Inverse reinforcement learning Suppose we are given an expert policy π_E that we wish to rationalize with IRL. For the remainder of this paper, we will adopt maximum causal entropy IRL [31, 32], which fits a cost function from a family of functions $\mathcal{C}$ with the optimization problem

$$\underset{c \in \mathcal{C}}{\text{maximize}} \left(\min_{\pi \in \Pi} -H(\pi) + \mathbb{E}_\pi[c(s, a)] \right) - \mathbb{E}_{\pi_E}[c(s, a)] \quad (1)$$

where $H(\pi) \triangleq \mathbb{E}_\pi[-\log \pi(a|s)]$ is the γ -discounted causal entropy [3] of the policy π . In practice, π_E will only be provided as a set of trajectories sampled by executing π_E in the environment, so the expected cost of π_E in Eq. (1) is estimated using these samples. Maximum causal entropy IRL looks for a cost function $c \in \mathcal{C}$ that assigns low cost to the expert policy and high cost to other policies, thereby allowing the expert policy to be found via a certain reinforcement learning procedure:

$$\text{RL}(c) = \arg \min_{\pi \in \Pi} -H(\pi) + \mathbb{E}_\pi[c(s, a)] \quad (2)$$

which maps a cost function to high-entropy policies that minimize the expected cumulative cost.

3 Characterizing the induced optimal policy

To begin our search for an imitation learning algorithm that both bypasses an intermediate IRL step and is suitable for large environments, we will study policies found by reinforcement learning on costs learned by IRL on the largest possible set of cost functions $\mathcal{C}$ in Eq. (1): *all* functions $\mathbb{R}^{\mathcal{S} \times \mathcal{A}} = \{c : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}\}$. Using expressive cost function classes, like Gaussian processes [15] and neural networks [7], is crucial to properly explain complex expert behavior without meticulously hand-crafted features. Here, we investigate the best IRL can do with respect to expressiveness, by examining its capabilities with $\mathcal{C} = \mathbb{R}^{\mathcal{S} \times \mathcal{A}}$.

Of course, with such a large $\mathcal{C}$, IRL can easily overfit when provided a finite dataset. Therefore, we will incorporate a (closed, proper) convex cost function regularizer $\psi : \mathbb{R}^{\mathcal{S} \times \mathcal{A}} \rightarrow \overline{\mathbb{R}}$ into our study. Note that convexity is a not particularly restrictive requirement: ψ must be convex as a function defined on all of $\mathbb{R}^{\mathcal{S} \times \mathcal{A}}$, not as a function defined on a small parameter space; indeed, the cost regularizers of Finn et al. [7], effective for a range of robotic manipulation tasks, satisfy this requirement. Interestingly, will in fact find that ψ plays a central role in our discussion, not a nuisance in our analysis.

Now, let us define an IRL primitive procedure, which finds a cost function such that the expert performs better than all other policies, with the cost regularized by ψ :

$$\text{IRL}_\psi(\pi_E) = \arg \max_{c \in \mathbb{R}^{\mathcal{S} \times \mathcal{A}}} -\psi(c) + \left(\min_{\pi \in \Pi} -H(\pi) + \mathbb{E}_\pi[c(s, a)] \right) - \mathbb{E}_{\pi_E}[c(s, a)] \quad (3)$$

Now let $\tilde{c} \in \text{IRL}_\psi(\pi_E)$. We are interested in a policy given by $\text{RL}(\tilde{c})$ —this is the policy given by running reinforcement learning on the output of IRL.

To characterize $\text{RL}(\tilde{c})$, it will be useful to transform optimization problems over policies into convex problems. For a policy $\pi \in \Pi$, define its occupancy measure $\rho_\pi : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ as $\rho_\pi(s, a) = \pi(a|s) \sum_{t=0}^{\infty} \gamma^t P(s_t = s | \pi)$. The occupancy measure can be interpreted as the distribution of state-action pairs that an agent encounters when navigating the environment with policy π , and it allows us to write $\mathbb{E}_\pi[c(s, a)] = \sum_{s,a} \rho_\pi(s, a) c(s, a)$ for any cost function c . A basic result [21] is that the set of valid occupancy measures $\mathcal{D} \triangleq \{\rho_\pi : \pi \in \Pi\}$ can be written as a feasible set of affine constraints: if $p_0(s)$ is the distribution of starting states and $P(s'|s, a)$ is the dynamics model, then $\mathcal{D} = \left\{ \rho : \rho \geq 0 \text{ and } \sum_a \rho(s, a) = p_0(s) + \gamma \sum_{s',a} P(s|s', a) \rho(s', a) \quad \forall s \in \mathcal{S} \right\}$. Furthermore, there is a one-to-one correspondence between Π and $\mathcal{D}$:

Proposition 3.1 (Theorem 2 of Syed et al. [29]). *If $\rho \in \mathcal{D}$, then ρ is the occupancy measure for $\pi_\rho(a|s) \triangleq \rho(s, a) / \sum_{a'} \rho(s, a')$, and π_ρ is the only policy whose occupancy measure is ρ .*

We are therefore justified in writing π_ρ to denote the unique policy for an occupancy measure ρ . We will need one more tool: for a function $f : \mathbb{R}^{\mathcal{S} \times \mathcal{A}} \rightarrow \mathbb{R}$, its convex conjugate $f^* : \mathbb{R}^{\mathcal{S} \times \mathcal{A}} \rightarrow \mathbb{R}$ is given by $f^*(x) = \sup_{y \in \mathbb{R}^{\mathcal{S} \times \mathcal{A}}} x^T y - f(y)$.

Now, we are ready to characterize $\text{RL}(\tilde{c})$, the policy learned by RL on the cost recovered by IRL:

Proposition 3.2. $\text{RL} \circ \text{IRL}_\psi(\pi_E) = \arg \min_{\pi \in \Pi} -H(\pi) + \psi^*(\rho_\pi - \rho_{\pi_E})$ (4)

The proof of Proposition 3.2 is in Appendix A.1. The proof relies on the observation that the optimal cost function and policy form a saddle point of a certain function. IRL finds one coordinate of this saddle point, and running reinforcement learning on the output of IRL reveals the other coordinate.

Proposition 3.2 tells us that ψ -regularized inverse reinforcement learning, implicitly, seeks a policy whose occupancy measure is close to the expert's, as measured by the convex function ψ^* . Enticingly, this suggests that various settings of ψ lead to various imitation learning algorithms that directly solve the optimization problem given by Proposition 3.2. We explore such algorithms in Sections 4 and 5, where we show that certain settings of ψ lead to both existing algorithms and a novel one.

The special case when ψ is a constant function is particularly illuminating, so we state and show it directly using concepts from convex optimization.

Corollary 3.2.1. *If ψ is a constant function, $\tilde{c} \in \text{IRL}_\psi(\pi_E)$, and $\tilde{\pi} \in \text{RL}(\tilde{c})$, then $\rho_{\tilde{\pi}} = \rho_{\pi_E}$.*

In other words, if there were no cost regularization at all, then the recovered policy will exactly match the expert's occupancy measure. To show this, we will need a lemma that lets us speak about causal entropies of occupancy measures:

Lemma 3.1. *Let $\bar{H}(\rho) = -\sum_{s,a} \rho(s, a) \log(\rho(s, a) / \sum_{a'} \rho(s, a'))$. Then, $\bar{H}$ is strictly concave, and for all $\pi \in \Pi$ and $\rho \in \mathcal{D}$, we have $H(\pi) = \bar{H}(\rho_\pi)$ and $\bar{H}(\rho) = H(\pi_\rho)$.*

The proof of this lemma is in Appendix A.1. Proposition 3.1 and Lemma 3.1 together allow us to freely switch between policies and occupancy measures when considering functions involving causal entropy and expected costs, as in the following lemma:

Lemma 3.2. *If $L(\pi, c) = -H(\pi) + \mathbb{E}_\pi[c(s, a)]$ and $\bar{L}(\rho, c) = -\bar{H}(\rho) + \sum_{s,a} \rho(s, a) c(s, a)$, then, for all cost functions c , $L(\pi, c) = \bar{L}(\rho_\pi, c)$ for all policies $\pi \in \Pi$, and $\bar{L}(\rho, c) = L(\pi_\rho, c)$ for all occupancy measures $\rho \in \mathcal{D}$.*

Now, we are ready to give a direct proof of Corollary 3.2.1

Proof of Corollary 3.2.1 Define $\bar{L}(\rho, c) = -\bar{H}(\rho) + \sum_{s,a} c(s, a) (\rho(s, a) - \rho_E(s, a))$. Given that ψ is a constant function, we have the following, due to Lemma 3.2:

$$\tilde{c} \in \text{IRL}_\psi(\pi_E) = \arg \max_{c \in \mathbb{R}^{\mathcal{S} \times \mathcal{A}}} \min_{\pi \in \Pi} -H(\pi) + \mathbb{E}_\pi[c(s, a)] - \mathbb{E}_{\pi_E}[c(s, a)] + \text{const.} \quad (5)$$

$$= \arg \max_{c \in \mathbb{R}^{\mathcal{S} \times \mathcal{A}}} \min_{\rho \in \mathcal{D}} -\bar{H}(\rho) + \sum_{s,a} \rho(s, a) c(s, a) - \sum_{s,a} \rho_E(s, a) c(s, a) = \arg \max_{c \in \mathbb{R}^{\mathcal{S} \times \mathcal{A}}} \min_{\rho \in \mathcal{D}} \bar{L}(\rho, c). \quad (6)$$

This is the dual of the optimization problem

$$\underset{\rho \in \mathcal{D}}{\text{minimize}} -\bar{H}(\rho) \quad \text{subject to} \quad \rho(s, a) = \rho_E(s, a) \quad \forall s \in \mathcal{S}, a \in \mathcal{A} \quad (7)$$

with Lagrangian $\bar{L}$, for which the costs $c(s, a)$ serve as dual variables for equality constraints. Thus, $\tilde{c}$ is a dual optimum for (7). Because $\mathcal{D}$ is a convex set and $-\bar{H}$ is convex, strong duality holds; moreover, Lemma 3.1 guarantees that $-\bar{H}$ is in fact strictly convex, so the primal optimum can be uniquely recovered from the dual optimum [4 Section 5.5.5] via $\tilde{\rho} = \arg \min_{\rho \in \mathcal{D}} \bar{L}(\rho, \tilde{c}) = \arg \min_{\rho \in \mathcal{D}} -\bar{H}(\rho) + \sum_{s,a} \tilde{c}(s, a) \rho(s, a) = \rho_E$, where the first equality indicates that $\tilde{\rho}$ is the unique minimizer of $\bar{L}(\cdot, \tilde{c})$, and the third follows from the constraints in the primal problem (7). But if $\tilde{\pi} \in \text{RL}(\tilde{c})$, then, by Lemma 3.2 its occupancy measure satisfies $\rho_{\tilde{\pi}} = \tilde{\rho} = \rho_E$. $\square$

From this argument, we can deduce the following:

IRL is a dual of an occupancy measure matching problem, and the recovered cost function is the dual optimum. Classic IRL algorithms that solve reinforcement learning repeatedly in an inner loop, such as the algorithm of Ziebart et al. [31] that runs a variant of value iteration in an inner loop, can be interpreted as a form of dual ascent, in which one repeatedly solves the primal problem (reinforcement learning) with fixed dual values (costs). Dual ascent is effective if solving the unconstrained primal is efficient, but in the case of IRL, it amounts to reinforcement learning!

The induced optimal policy is the primal optimum. The induced optimal policy is obtained by running RL after IRL, which is exactly the act of recovering the primal optimum from the dual optimum; that is, optimizing the Lagrangian with the dual variables fixed at the dual optimum values. Strong duality implies that this induced optimal policy is indeed the primal optimum, and therefore matches occupancy measures with the expert. IRL is traditionally defined as the act of finding a cost function such that the expert policy is uniquely optimal, but now, we can alternatively view IRL as a procedure that tries to *induce a policy that matches the expert's occupancy measure*.

4 Practical occupancy measure matching

We saw in Corollary 3.2.1 that if ψ is constant, the resulting primal problem (7) simply matches occupancy measures with expert at all states and actions. Such an algorithm, however, is not practically useful. In reality, the expert trajectory distribution will be provided only as a finite set of samples, so in large environments, most of the expert's occupancy measure values will be exactly zero, and exact occupancy measure matching will force the learned policy to never visit these unseen state-action pairs simply due to lack of data. Furthermore, with large environments, we would like to use function approximation to learn a parameterized policy π_θ . The resulting optimization problem of finding the appropriate θ would have as many constraints as points in $\mathcal{S} \times \mathcal{A}$, leading to an intractably large problem and defeating the very purpose of function approximation.

Keeping in mind that we wish to eventually develop an imitation learning algorithm suitable for large environments, we would like to relax Eq. (7) into the following form, motivated by Proposition 3.2

$$\underset{\pi}{\text{minimize}} \quad d_\psi(\rho_\pi, \rho_E) - H(\pi) \quad (8)$$

by modifying the IRL regularizer ψ so that $d_\psi(\rho_\pi, \rho_E) \triangleq \psi^*(\rho_\pi - \rho_E)$ smoothly penalizes violations in difference between the occupancy measures.

Entropy-regularized apprenticeship learning It turns out that with certain settings of ψ , Eq. (8) takes on the form of regularized variants of existing *apprenticeship learning* algorithms, which indeed do scale to large environments with parameterized policies [11]. For a class of cost functions $\mathcal{C} \subset \mathbb{R}^{\mathcal{S} \times \mathcal{A}}$, an apprenticeship learning algorithm finds a policy that performs better than the expert across $\mathcal{C}$, by optimizing the objective

$$\underset{\pi}{\text{minimize}} \quad \max_{c \in \mathcal{C}} \mathbb{E}_\pi[c(s, a)] - \mathbb{E}_{\pi_E}[c(s, a)] \quad (9)$$

Classic apprenticeship learning algorithms restrict $\mathcal{C}$ to convex sets given by linear combinations of basis functions $f_1, \dots, f_d$, which give rise a feature vector $f(s, a) = [f_1(s, a), \dots, f_d(s, a)]$ for each state-action pair. Abbeel and Ng [1] and Syed et al. [29] use, respectively,

$$\mathcal{C}_{\text{linear}} = \{\sum_i w_i f_i : \|w\|_2 \leq 1\} \quad \text{and} \quad \mathcal{C}_{\text{convex}} = \{\sum_i w_i f_i : \sum_i w_i = 1, w_i \geq 0 \forall i\}. \quad (10)$$

$\mathcal{C}_{\text{linear}}$ leads to feature expectation matching [1], which minimizes ℓ_2 distance between expected feature vectors: $\max_{c \in \mathcal{C}_{\text{linear}}} \mathbb{E}_{\pi} [c(s, a)] - \mathbb{E}_{\pi_E} [c(s, a)] = \|\mathbb{E}_{\pi} [f(s, a)] - \mathbb{E}_{\pi_E} [f(s, a)]\|_2$. Meanwhile, $\mathcal{C}_{\text{convex}}$ leads to MWAL [28] and LPAL [29], which minimize worst-case excess cost among the individual basis functions, as $\max_{c \in \mathcal{C}_{\text{convex}}} \mathbb{E}_{\pi} [c(s, a)] - \mathbb{E}_{\pi_E} [c(s, a)] = \max_{i \in \{1, \dots, d\}} \mathbb{E}_{\pi} [f_i(s, a)] - \mathbb{E}_{\pi_E} [f_i(s, a)]$.

We now show how Eq. (9) is a special case of Eq. (8) with a certain setting of ψ . With the indicator function $\delta_{\mathcal{C}} : \mathbb{R}^{S \times A} \rightarrow \mathbb{R}$, defined by $\delta_{\mathcal{C}}(c) = 0$ if $c \in \mathcal{C}$ and $+\infty$ otherwise, we can write the apprenticeship learning objective (9) as

$$\max_{c \in \mathcal{C}} \mathbb{E}_{\pi} [c(s, a)] - \mathbb{E}_{\pi_E} [c(s, a)] = \max_{c \in \mathbb{R}^{S \times A}} -\delta_{\mathcal{C}}(c) + \sum_{s, a} (\rho_{\pi}(s, a) - \rho_{\pi_E}(s, a)) c(s, a) = \delta_{\mathcal{C}}^*(\rho_{\pi} - \rho_{\pi_E})$$

Therefore, we see that entropy-regularized apprenticeship learning

$$\underset{\pi}{\text{minimize}} -H(\pi) + \max_{c \in \mathcal{C}} \mathbb{E}_{\pi} [c(s, a)] - \mathbb{E}_{\pi_E} [c(s, a)] \quad (11)$$

is equivalent to performing RL following IRL with cost regularizer $\psi = \delta_{\mathcal{C}}$, which forces the implicit IRL procedure to recover a cost function lying in $\mathcal{C}$. Note that we can scale the policy’s entropy regularization strength in Eq. (11) by scaling $\mathcal{C}$ by a constant α as $\{\alpha c : c \in \mathcal{C}\}$, recovering the original apprenticeship objective (9) by taking $\alpha \rightarrow \infty$.

Cons of apprenticeship learning It is known that apprenticeship learning algorithms generally do not recover expert-like policies if $\mathcal{C}$ is too restrictive [29, Section 1]—which is often the case for the linear subspaces used by feature expectation matching, MWAL, and LPAL, unless the basis functions $f_1, \dots, f_d$ are very carefully designed. Intuitively, unless the true expert cost function (assuming it exists) lies in $\mathcal{C}$, there is no guarantee that if π performs better than π_E on all of $\mathcal{C}$, then π equals π_E . With the aforementioned insight based on Proposition 3.2 that apprenticeship learning is equivalent to RL following IRL, we can understand exactly why apprenticeship learning may fail to imitate: it forces π_E to be encoded as an element of $\mathcal{C}$. If $\mathcal{C}$ does not include a cost function that explains expert behavior well, then attempting to recover a policy from such an encoding will not succeed.

Pros of apprenticeship learning While restrictive cost classes $\mathcal{C}$ may not lead to exact imitation, apprenticeship learning with such $\mathcal{C}$ can scale to large state and action spaces with policy function approximation. Ho et al. [11] rely on the following policy gradient formula for the apprenticeship objective (9) for a parameterized policy π_{θ} :

$$\nabla_{\theta} \max_{c \in \mathcal{C}} \mathbb{E}_{\pi_{\theta}} [c(s, a)] - \mathbb{E}_{\pi_E} [c(s, a)] = \nabla_{\theta} \mathbb{E}_{\pi_{\theta}} [c^*(s, a)] = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a|s) Q_{c^*}(s, a)] \quad (12)$$

where $c^* = \arg \max_{c \in \mathcal{C}} \mathbb{E}_{\pi_{\theta}} [c(s, a)] - \mathbb{E}_{\pi_E} [c(s, a)]$, $Q_{c^*}(\bar{s}, \bar{a}) = \mathbb{E}_{\pi_{\theta}} [c^*(\bar{s}, \bar{a}) | s_0 = \bar{s}, a_0 = \bar{a}]$

Observing that Eq. (12) is the policy gradient for a reinforcement learning objective with cost c^* , [Ho et al.] propose an algorithm that alternates between two steps:

1. Sample trajectories of the current policy π_{θ_i} by simulating in the environment, and fit a cost function c_i^* , as defined in Eq. (12). For the cost classes $\mathcal{C}_{\text{linear}}$ and $\mathcal{C}_{\text{convex}}$ [10], this cost fitting amounts to evaluating simple analytical expressions [11].
2. Form a gradient estimate with Eq. (12) with c_i^* and the sampled trajectories, and take a trust region policy optimization (TRPO) [26] step to produce $\pi_{\theta_{i+1}}$.

This algorithm relies crucially on the TRPO policy step, which is a natural gradient step constrained to ensure that $\pi_{\theta_{i+1}}$ does not stray too far from π_{θ_i} , as measured by KL divergence between the two policies averaged over the states in the sampled trajectories. This carefully constructed step scheme ensures that divergence does not occur due to high noise in estimating the gradient [12]. We refer the reader to Schulman et al. [26] for more details on TRPO.

With the TRPO step scheme, [Ho et al.] were able to train large neural network policies for apprenticeship learning with linear cost function classes [10] in environments with hundreds of observation dimensions. Their use of these linear cost function classes, however, limits their approach to settings in which expert behavior is well-described by such classes. We will draw upon their algorithm to develop an imitation learning method that both scales to large environments and imitates arbitrarily complex expert behavior. To do so, we first turn to proposing a new regularizer ψ that yields more expressive power than the regularizers corresponding to $\mathcal{C}_{\text{linear}}$ and $\mathcal{C}_{\text{convex}}$ [10].

5 Generative adversarial imitation learning

As discussed in Section 4, the constant regularizer leads to an imitation learning algorithm that exactly matches occupancy measures, but is intractable in large environments. The indicator regularizers for the linear cost function classes (10), on the other hand, lead to algorithms incapable of exactly matching occupancy measures without careful tuning, but are tractable in large environments. We propose the following new cost regularizer that combines the best of both worlds, as we will show in the coming sections:

$$\psi_{\text{GA}}(c) \triangleq \begin{cases} \mathbb{E}_{\pi_E}[g(c(s, a))] & \text{if } c < 0 \\ +\infty & \text{otherwise} \end{cases} \quad \text{where } g(x) = \begin{cases} -x - \log(1 - e^x) & \text{if } x < 0 \\ +\infty & \text{otherwise} \end{cases} \quad (13)$$

This regularizer places low penalty on cost functions c that assign an amount of negative cost to expert state-action pairs; if c , however, assigns large costs (close to zero, which is the upper bound for costs feasible for ψ_{GA}) to the expert, then ψ_{GA} will heavily penalize c . An interesting property of ψ_{GA} is that it is an average over expert data, and therefore can adjust to arbitrary expert datasets. The indicator regularizers δ_C , used by the linear apprenticeship learning algorithms described in Section 4, are always fixed, and cannot adapt to data as ψ_{GA} can. Perhaps the most important difference between ψ_{GA} and δ_C , however, is that δ_C forces costs to lie in a small subspace spanned by finitely many basis functions, whereas ψ_{GA} allows for any cost function, as long as it is negative everywhere.

Our choice of ψ_{GA} is motivated by the following fact, shown in the appendix (Corollary A.1.1):

$$\psi_{\text{GA}}^*(\rho_\pi - \rho_{\pi_E}) = \max_{D \in (0,1)^{\mathcal{S} \times \mathcal{A}}} \mathbb{E}_\pi[\log(D(s, a))] + \mathbb{E}_{\pi_E}[\log(1 - D(s, a))] \quad (14)$$

where the maximum ranges over discriminative classifiers $D : \mathcal{S} \times \mathcal{A} \rightarrow (0, 1)$. Equation (14) is the optimal negative log loss of the binary classification problem of distinguishing between state-action pairs of π and π_E . It turns out that this optimal loss is (up to a constant shift) the Jensen-Shannon divergence $D_{\text{JS}}(\rho_\pi, \rho_{\pi_E}) \triangleq D_{\text{KL}}(\rho_\pi \| (\rho_\pi + \rho_E)/2) + D_{\text{KL}}(\rho_E \| (\rho_\pi + \rho_E)/2)$, which is a squared metric between distributions [9, 19]. Treating the causal entropy H as a policy regularizer, controlled by $\lambda \geq 0$, we obtain a new imitation learning algorithm:

$$\underset{\pi}{\text{minimize}} \quad \psi_{\text{GA}}^*(\rho_\pi - \rho_{\pi_E}) - \lambda H(\pi) = D_{\text{JS}}(\rho_\pi, \rho_{\pi_E}) - \lambda H(\pi), \quad (15)$$

which finds a policy whose occupancy measure minimizes Jensen-Shannon divergence to the expert's. Equation (15) minimizes a true metric between occupancy measures, so, unlike linear apprenticeship learning algorithms, it can imitate expert policies exactly.

Algorithm Equation (15) draws a connection between imitation learning and generative adversarial networks [9], which train a generative model G by having it confuse a discriminative classifier D . The job of D is to distinguish between the distribution of data generated by G and the true data distribution. When D cannot distinguish data generated by G from the true data, then G has successfully matched the true data. In our setting, the learner's occupancy measure ρ_π is analogous to the data distribution generated by G , and the expert's occupancy measure ρ_{π_E} is analogous to the true data distribution.

Now, we present a practical algorithm, which we call *generative adversarial imitation learning* (Algorithm 1), for solving Eq. (15) for model-free imitation in large environments. Explicitly, we wish to find a saddle point (π, D) of the expression

$$\mathbb{E}_\pi[\log(D(s, a))] + \mathbb{E}_{\pi_E}[\log(1 - D(s, a))] - \lambda H(\pi) \quad (16)$$

To do so, we first introduce function approximation for π and D : we will fit a parameterized policy π_θ , with weights θ , and a discriminator network $D_w : \mathcal{S} \times \mathcal{A} \rightarrow (0, 1)$, with weights w . Then, we alternate between an Adam [12] gradient step on w to increase Eq. (16) with respect to D , and a TRPO step on θ to decrease Eq. (16) with respect to π . The TRPO step serves the same purpose as it does with the apprenticeship learning algorithm of Ho et al. [11]: it prevents the policy from changing too much due to noise in the policy gradient. The discriminator network can be interpreted as a local cost function providing learning signal to the policy—specifically, taking a policy step that decreases expected cost with respect to the cost function $c(s, a) = \log D(s, a)$ will move toward expert-like regions of state-action space, as classified by the discriminator. (We derive an estimator for the causal entropy gradient $\nabla_\theta H(\pi_\theta)$ in Appendix A.2)

Algorithm 1 Generative adversarial imitation learning

- 1: **Input:** Expert trajectories $\tau_E \sim \pi_E$, initial policy and discriminator parameters θ_0, w_0
- 2: **for** $i = 0, 1, 2, \dots$ **do**
- 3: Sample trajectories $\tau_i \sim \pi_{\theta_i}$
- 4: Update the discriminator parameters from w_i to w_{i+1} with the gradient

$$\hat{\mathbb{E}}_{\tau_i}[\nabla_w \log(D_w(s, a))] + \hat{\mathbb{E}}_{\tau_E}[\nabla_w \log(1 - D_w(s, a))] \quad (17)$$

- 5: Take a policy step from θ_i to θ_{i+1} , using the TRPO rule with cost function $\log(D_{w_{i+1}}(s, a))$. Specifically, take a KL-constrained natural gradient step with

$$\hat{\mathbb{E}}_{\tau_i}[\nabla_{\theta} \log \pi_{\theta}(a|s) Q(s, a)] - \lambda \nabla_{\theta} H(\pi_{\theta}), \quad (18)$$

where $Q(\bar{s}, \bar{a}) = \hat{\mathbb{E}}_{\tau_i}[\log(D_{w_{i+1}}(s, a)) \mid s_0 = \bar{s}, a_0 = \bar{a}]$

- 6: **end for**
-

6 Experiments

We evaluated Algorithm 1 against baselines on 9 physics-based control tasks, ranging from low-dimensional control tasks from the classic RL literature—the cartpole [2], acrobot [8], and mountain car [17]—to difficult high-dimensional tasks such as a 3D humanoid locomotion, solved only recently by model-free reinforcement learning [27, 26]. All environments, other than the classic control tasks, were simulated with MuJoCo [30]. See Appendix B for a complete description of all the tasks.

Each task comes with a true cost function, defined in the OpenAI Gym [5]. We first generated expert behavior for these tasks by running TRPO [26] on these true cost functions to create expert policies. Then, to evaluate imitation performance with respect to sample complexity of expert data, we sampled datasets of varying trajectory counts from the expert policies. The trajectories constituting each dataset each consisted of about 50 state-action pairs. We tested Algorithm 1 against three baselines:

1. Behavioral cloning: a given dataset of state-action pairs is split into 70% training data and 30% validation data. The policy is trained with supervised learning, using Adam [12] with minibatches of 128 examples, until validation error stops decreasing.
2. Feature expectation matching (FEM): the algorithm of Ho et al. [11] using the cost function class $\mathcal{C}_{\text{linear}}$ [10] of Abbeel and Ng [1]
3. Game-theoretic apprenticeship learning (GTAL): the algorithm of Ho et al. [11] using the cost function class $\mathcal{C}_{\text{convex}}$ [10] of Syed and Schapire [28]

We used all algorithms to train policies of the same neural network architecture for all tasks: two hidden layers of 100 units each, with tanh nonlinearities in between. The discriminator networks for Algorithm 1 also used the same architecture. All networks were always initialized randomly at the start of each trial. For each task, we gave FEM, GTAL, and Algorithm 1 exactly the same amount of environment interaction for training.

Figure 1 depicts the results, and the tables in Appendix B provide exact performance numbers. We found that on the classic control tasks (cartpole, acrobot, and mountain car), behavioral cloning suffered in expert data efficiency compared to FEM and GTAL, which for the most part were able to produce policies with near-expert performance with a wide range of dataset sizes. On these tasks, our generative adversarial algorithm always produced policies performing better than behavioral cloning, FEM, and GTAL. However, behavioral cloning performed excellently on the Reacher task, on which it was more sample efficient than our algorithm. We were able to slightly improve our algorithm’s performance on Reacher using causal entropy regularization—in the 4-trajectory setting, the improvement from $\lambda = 0$ to $\lambda = 10^{-3}$ was statistically significant over training reruns, according to a one-sided Wilcoxon rank-sum test with $p = .05$. We used no causal entropy regularization for all other tasks.

On the other MuJoCo environments, we saw a large performance boost for our algorithm over the baselines. Our algorithm almost always achieved at least 70% of expert performance for all dataset

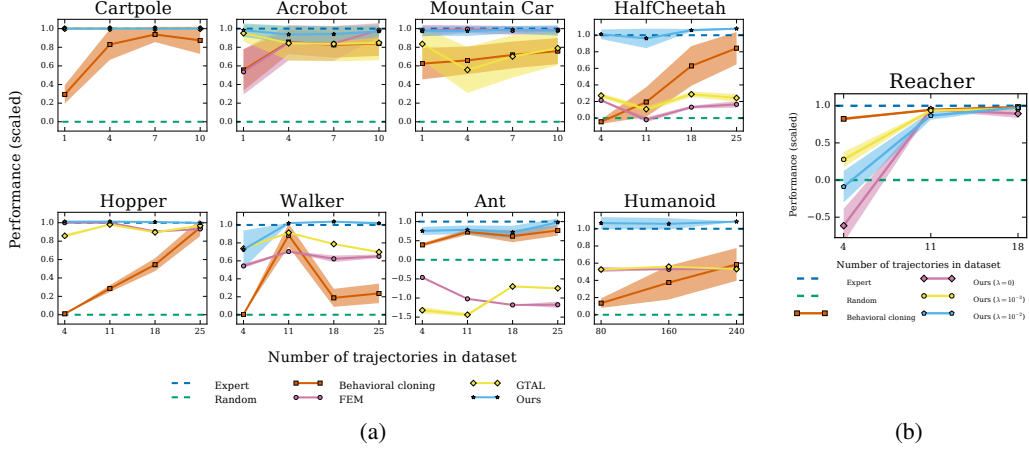


Figure 1: (a) Performance of learned policies. The y -axis is negative cost, scaled so that the expert achieves 1 and a random policy achieves 0. (b) Causal entropy regularization λ on Reacher.

sizes we tested, nearly always dominating all the baselines. FEM and GTAL performed poorly for Ant, producing policies consistently worse than a policy that chooses actions uniformly at random. Behavioral cloning was able to reach satisfactory performance with enough data on HalfCheetah, Hopper, Walker, and Ant; but was unable to achieve more than 60% for Humanoid, on which our algorithm achieved exact expert performance for all tested dataset sizes.

7 Discussion and outlook

As we demonstrated, our method is generally quite sample efficient in terms of expert data. However, it is not particularly sample efficient in terms of environment interaction during training. The number of such samples required to estimate the imitation objective gradient (18) was comparable to the number needed for TRPO to train the expert policies from reinforcement signals. We believe that we could significantly improve learning speed for our algorithm by initializing policy parameters with behavioral cloning, which requires no environment interaction at all.

Fundamentally, our method is model free, so it will generally need more environment interaction than model-based methods. Guided cost learning [7], for instance, builds upon guided policy search [13] and inherits its sample efficiency, but also inherits its requirement that the model is well-approximated by iteratively fitted time-varying linear dynamics. Interestingly, both our Algorithm 1 and guided cost learning alternate between policy optimization steps and cost fitting (which we called discriminator fitting), even though the two algorithms are derived completely differently.

Our approach builds upon a vast line of work on IRL [31, 1, 29, 28], and hence, just like IRL, our approach does not interact with the expert during training. Our method explores randomly to determine which actions bring a policy’s occupancy measure closer to the expert’s, whereas methods that do interact with the expert, like DAgger [24], can simply ask the expert for such actions. Ultimately, we believe that a method that combines well-chosen environment models with expert interaction will win in terms of sample complexity of both expert data and environment interaction.

Acknowledgments

We thank Jayesh K. Gupta and John Schulman for assistance and advice. This work was supported by the SAIL-Toyota Center for AI Research, and by a NSF Graduate Research Fellowship (grant no. DGE-114747).

References

- [1] P. Abbeel and A. Y. Ng. Apprenticeship learning via inverse reinforcement learning. In *Proceedings of the 21st International Conference on Machine Learning*, 2004.
- [2] A. G. Barto, R. S. Sutton, and C. W. Anderson. Neuronlike adaptive elements that can solve difficult learning control problems. *Systems, Man and Cybernetics, IEEE Transactions on*, (5):834–846, 1983.

- [3] M. Bloem and N. Bambos. Infinite time horizon maximum causal entropy inverse reinforcement learning. In *Decision and Control (CDC), 2014 IEEE 53rd Annual Conference on*, pages 4911–4916. IEEE, 2014.
- [4] S. Boyd and L. Vandenberghe. *Convex optimization*. Cambridge university press, 2004.
- [5] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba. OpenAI Gym. *arXiv preprint arXiv:1606.01540*, 2016.
- [6] T. M. Cover and J. A. Thomas. *Elements of information theory*. John Wiley & Sons, 2012.
- [7] C. Finn, S. Levine, and P. Abbeel. Guided cost learning: Deep inverse optimal control via policy optimization. In *Proceedings of the 33rd International Conference on Machine Learning*, 2016.
- [8] A. Geramifard, C. Dann, R. H. Klein, W. Dabney, and J. P. How. Rlpy: A value-function-based reinforcement learning framework for education and research. *JMLR*, 2015.
- [9] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. Generative adversarial nets. In *NIPS*, pages 2672–2680, 2014.
- [10] J.-B. Hiriart-Urruty and C. Lemaréchal. *Convex Analysis and Minimization Algorithms*, volume 305. Springer, 1996.
- [11] J. Ho, J. K. Gupta, and S. Ermon. Model-free imitation learning with policy optimization. In *Proceedings of the 33rd International Conference on Machine Learning*, 2016.
- [12] D. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [13] S. Levine and P. Abbeel. Learning neural network policies with guided policy search under unknown dynamics. In *Advances in Neural Information Processing Systems*, pages 1071–1079, 2014.
- [14] S. Levine and V. Koltun. Continuous inverse optimal control with locally optimal examples. In *Proceedings of the 29th International Conference on Machine Learning*, pages 41–48, 2012.
- [15] S. Levine, Z. Popovic, and V. Koltun. Nonlinear inverse reinforcement learning with gaussian processes. In *Advances in Neural Information Processing Systems*, pages 19–27, 2011.
- [16] P. W. Millar. The minimax principle in asymptotic statistical theory. In *Ecole d’Eté de Probabilités de Saint-Flour XI—1981*, pages 75–265. Springer, 1983.
- [17] A. W. Moore and T. Hall. Efficient memory-based learning for robot control. 1990.
- [18] A. Y. Ng and S. Russell. Algorithms for inverse reinforcement learning. In *ICML*, 2000.
- [19] X. Nguyen, M. J. Wainwright, and M. I. Jordan. On surrogate loss functions and f-divergences. *The Annals of Statistics*, pages 876–904, 2009.
- [20] D. A. Pomerleau. Efficient training of artificial neural networks for autonomous navigation. *Neural Computation*, 3(1):88–97, 1991.
- [21] M. L. Puterman. *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 2014.
- [22] N. D. Ratliff, D. Silver, and J. A. Bagnell. Learning to search: Functional gradient techniques for imitation learning. *Autonomous Robots*, 27(1):25–53, 2009.
- [23] S. Ross and D. Bagnell. Efficient reductions for imitation learning. In *AISTATS*, pages 661–668, 2010.
- [24] S. Ross, G. J. Gordon, and D. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *AISTATS*, pages 627–635, 2011.
- [25] S. Russell. Learning agents for uncertain environments. In *Proceedings of the Eleventh Annual Conference on Computational Learning Theory*, pages 101–103. ACM, 1998.
- [26] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz. Trust region policy optimization. In *Proceedings of The 32nd International Conference on Machine Learning*, pages 1889–1897, 2015.
- [27] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.
- [28] U. Syed and R. E. Schapire. A game-theoretic approach to apprenticeship learning. In *Advances in Neural Information Processing Systems*, pages 1449–1456, 2007.
- [29] U. Syed, M. Bowling, and R. E. Schapire. Apprenticeship learning using linear programming. In *Proceedings of the 25th International Conference on Machine Learning*, pages 1032–1039, 2008.
- [30] E. Todorov, T. Erez, and Y. Tassa. Mujoco: A physics engine for model-based control. In *Intelligent Robots and Systems (IROS), 2012 IEEE/RSJ International Conference on*, pages 5026–5033. IEEE, 2012.
- [31] B. D. Ziebart, A. Maas, J. A. Bagnell, and A. K. Dey. Maximum entropy inverse reinforcement learning. In *AAAI, AAAI’08*, 2008.
- [32] B. D. Ziebart, J. A. Bagnell, and A. K. Dey. Modeling interaction via the principle of maximum causal entropy. In *ICML*, pages 1255–1262, 2010.

A Proofs

A.1 Proofs for Section 3

Proof of Lemma 3.1 First, we show strict concavity of $\bar{H}$. Let ρ and ρ' be occupancy measures, and suppose $\lambda \in [0, 1]$. For all s and a , the log-sum inequality [6] implies:

$$-(\lambda\rho(s, a) + (1 - \lambda)\rho'(s, a)) \log \frac{\lambda\rho(s, a) + (1 - \lambda)\rho'(s, a)}{\sum_{a'} (\lambda\rho(s, a') + (1 - \lambda)\rho'(s, a'))} \quad (19)$$

$$= -(\lambda\rho(s, a) + (1 - \lambda)\rho'(s, a)) \log \frac{\lambda\rho(s, a) + (1 - \lambda)\rho'(s, a)}{\lambda \sum_{a'} \rho(s, a') + (1 - \lambda) \sum_{a'} \rho'(s, a')} \quad (20)$$

$$\geq -\lambda\rho(s, a) \log \frac{\lambda\rho(s, a)}{\lambda \sum_{a'} \rho(s, a')} - (1 - \lambda)\rho'(s, a) \log \frac{(1 - \lambda)\rho'(s, a)}{(1 - \lambda) \sum_{a'} \rho'(s, a')} \quad (21)$$

$$= \lambda \left(-\rho(s, a) \log \frac{\rho(s, a)}{\sum_{a'} \rho(s, a')} \right) + (1 - \lambda) \left(-\rho'(s, a) \log \frac{\rho'(s, a)}{\sum_{a'} \rho'(s, a')} \right), \quad (22)$$

with equality if and only if $\pi_\rho \triangleq \rho(s, a) / \sum_{a'} \rho(s, a') = \rho'(s, a) / \sum_{a'} \rho'(s, a') \triangleq \pi_{\rho'}$. Summing both sides over all s and a shows that $\bar{H}(\lambda\rho + (1 - \lambda)\rho') \geq \lambda\bar{H}(\rho) + (1 - \lambda)\bar{H}(\rho')$ with equality if and only if $\pi_\rho = \pi_{\rho'}$. Applying Proposition 3.1 shows that equality in fact holds if and only if $\rho = \rho'$, so $\bar{H}$ is strictly concave.

Now, we turn to verifying the last two statements, which also follow from Proposition 3.1 and the definition of occupancy measures. First,

$$H(\pi) = \mathbb{E}_\pi[-\log \pi(a|s)] \quad (23)$$

$$= -\sum_{s,a} \rho_\pi(s, a) \log \pi(a|s) \quad (24)$$

$$= -\sum_{s,a} \rho_\pi(s, a) \log \frac{\rho_\pi(s, a)}{\sum_{a'} \rho_\pi(s, a')} \quad (25)$$

$$= \bar{H}(\rho_\pi), \quad (26)$$

and second,

$$\bar{H}(\rho) = -\sum_{s,a} \rho(s, a) \log \frac{\rho(s, a)}{\sum_{a'} \rho(s, a')} \quad (27)$$

$$= -\sum_{s,a} \rho_{\pi_\rho}(s, a) \log \pi_\rho(a|s) \quad (28)$$

$$= \mathbb{E}_{\pi_\rho}[-\log \pi_\rho(a|s)] \quad (29)$$

$$= H(\pi_\rho). \quad (30)$$

□

Proof of Proposition 3.2 This proof relies on properties of saddle points. For a reference, we refer the reader to Hiriart-Urruty and Lemaréchal [10, section VII.4].

Let $\tilde{c} \in \text{IRL}_\psi(\pi_E)$, $\tilde{\pi} \in \text{RL}(\tilde{c}) = \text{RL} \circ \text{IRL}_\psi(\pi_E)$, and

$$\pi_A \in \arg \min_{\pi} -H(\pi) + \psi^*(\rho_\pi - \rho_{\pi_E}) \quad (31)$$

$$= \arg \min_{\pi} \max_c -H(\pi) - \psi(c) + \sum_{s,a} (\rho_\pi(s, a) - \rho_{\pi_E}(s, a))c(s, a) \quad (32)$$

We wish to show that $\pi_A = \tilde{\pi}$. To do this, let ρ_A be the occupancy measure of π_A , let $\tilde{\rho}$ be the occupancy measure of $\tilde{\pi}$, and define $\bar{L} : \mathcal{D} \times \mathbb{R}^{S \times \mathcal{A}} \rightarrow \mathbb{R}$ by

$$\bar{L}(\rho, c) = -\bar{H}(\rho) - \psi(c) + \sum_{s,a} \rho(s, a)c(s, a) - \sum_{s,a} \rho_{\pi_E}(s, a)c(s, a). \quad (33)$$

The following relationships then hold, due to Proposition 3.1

$$\rho_A \in \arg \min_{\rho \in \mathcal{D}} \max_c \bar{L}(\rho, c), \quad (34)$$

$$\tilde{c} \in \arg \max_c \min_{\rho \in \mathcal{D}} \bar{L}(\rho, c), \quad (35)$$

$$\tilde{\rho} \in \arg \min_{\rho \in \mathcal{D}} \bar{L}(\rho, \tilde{c}). \quad (36)$$

Now $\mathcal{D}$ is compact and convex and $\mathbb{R}^{S \times A}$ is convex; furthermore, due to convexity of $-\bar{H}$ and ψ , we also have that $\bar{L}(\cdot, c)$ is convex for all c , and that $\bar{L}(\rho, \cdot)$ is concave for all ρ . Therefore, we can use minimax duality 16:

$$\min_{\rho \in \mathcal{D}} \max_{c \in \mathcal{C}} \bar{L}(\rho, c) = \max_{c \in \mathcal{C}} \min_{\rho \in \mathcal{D}} \bar{L}(\rho, c) \quad (37)$$

Hence, from Eqs. 34 and 35, $(\rho_A, \tilde{c})$ is a saddle point of $\bar{L}$, which implies that

$$\rho_A \in \arg \min_{\rho \in \mathcal{D}} \bar{L}(\rho, \tilde{c}). \quad (38)$$

Because $\bar{L}(\cdot, c)$ is strictly convex for all c (Lemma 3.1), Eqs. 36 and 38 imply $\rho_A = \tilde{\rho}$. Since policies corresponding to occupancy measures are unique (Proposition 3.1), we get $\pi_A = \tilde{\pi}$. $\square$

A.2 Proofs for Section 5

In Eq. 13 of Section 5 we described a cost regularizer ψ_{GA} , which leads to an imitation learning algorithm 15 that minimizes Jensen-Shannon divergence between occupancy measures. To justify our choice of ψ_{GA} , we show how to convert certain surrogate loss functions ϕ , for binary classification of state-action pairs drawn from the occupancy measures ρ_π and ρ_{π_E} , into cost function regularizers ψ , for which $\psi^*(\rho_\pi - \rho_{\pi_E})$ is the minimum expected risk $R_\phi(\rho_\pi, \rho_{\pi_E})$ for ϕ :

$$R_\phi(\pi, \pi_E) = \sum_{s,a} \min_{\gamma \in \mathbb{R}} \rho_\pi(s, a) \phi(\gamma) + \rho_{\pi_E}(s, a) \phi(-\gamma) \quad (39)$$

Specifically, we will restrict ourselves to strictly decreasing convex loss functions. Nguyen et al. 19 show a correspondence between minimum expected risks R_ϕ and f -divergences, of which Jensen-Shannon divergence is a special case. Our following construction, therefore, can generate any imitation learning algorithm that minimizes an f -divergence between occupancy measures, as long as that f -divergence is induced by a strictly decreasing convex surrogate ϕ .

Proposition A.1. Suppose $\phi : \mathbb{R} \rightarrow \mathbb{R}$ is a strictly decreasing convex function. Let T be the range of $-\phi$, and define $g_\phi : \mathbb{R} \rightarrow \mathbb{R}$ and $\psi_\phi : \mathbb{R}^{S \times A} \rightarrow \mathbb{R}$ by:

$$\begin{aligned} g_\phi(x) &= \begin{cases} -x + \phi(-\phi^{-1}(-x)) & \text{if } x \in T \\ +\infty & \text{otherwise} \end{cases} \\ \psi_\phi(c) &= \begin{cases} \sum_{s,a} \rho_{\pi_E}(s, a) g_\phi(c(s, a)) & \text{if } c(s, a) \in T \text{ for all } s, a \\ +\infty & \text{otherwise} \end{cases} \end{aligned} \quad (40)$$

Then, ψ_ϕ is closed, proper, and convex, and $\text{RL} \circ \text{IRL}_{\psi_\phi}(\pi_E) = \arg \min_{\pi} -H(\pi) - R_\phi(\rho_\pi, \rho_{\pi_E})$.

Proof. To verify the first claim, it suffices to check that $g_\phi(x) = -x + \phi(-\phi^{-1}(-x))$ is closed, proper, and convex. Convexity follows from the fact that $x \mapsto \phi(-\phi^{-1}(-x))$ is convex, because it is a concave function followed by a nonincreasing convex function. Furthermore, because T is nonempty, g_ϕ is proper. To show that g_ϕ is closed, note that because ϕ is strictly decreasing and convex, the range of ϕ is either all of $\mathbb{R}$ or an open interval (b, ∞) for some $b \in \mathbb{R}$. If the range of ϕ is $\mathbb{R}$, then g_ϕ is finite everywhere and is therefore closed. On the other hand, if the range of ϕ is (b, ∞) , then $\phi(x) \rightarrow b$ as $x \rightarrow \infty$, and $\phi(x) \rightarrow \infty$ as $x \rightarrow -\infty$. Thus, as $x \rightarrow b$, $\phi^{-1}(-x) \rightarrow \infty$, so $\phi(-\phi^{-1}(-x)) \rightarrow \infty$ too, implying that $g_\phi(x) \rightarrow \infty$ as $x \rightarrow b$, which means g_ϕ is closed.

Now, we verify the second claim. By Proposition 3.2, all we need to check is that $-R_\phi(\rho_\pi, \rho_{\pi_E}) = \psi_\phi^*(\rho_\pi - \rho_{\pi_E})$:

$$\psi_\phi^*(\rho_\pi - \rho_{\pi_E}) = \max_{c \in \mathcal{C}} \sum_{s,a} (\rho_\pi(s,a) - \rho_{\pi_E}(s,a))c(s,a) - \sum_{s,a} \rho_{\pi_E}(s,a)g_\phi(c(s,a)) \quad (41)$$

$$= \sum_{s,a} \max_{c \in T} (\rho_\pi(s,a) - \rho_{\pi_E}(s,a))c - \rho_{\pi_E}(s,a)[-c + \phi(-\phi^{-1}(-c))] \quad (42)$$

$$= \sum_{s,a} \max_{c \in T} \rho_\pi(s,a)c - \rho_{\pi_E}(s,a)\phi(-\phi^{-1}(-c)) \quad (43)$$

$$= \sum_{s,a} \max_{\gamma \in \mathbb{R}} \rho_\pi(s,a)(-\phi(\gamma)) - \rho_{\pi_E}(s,a)\phi(-\phi^{-1}(\phi(\gamma))) \quad (44)$$

$$= \sum_{s,a} \max_{\gamma \in \mathbb{R}} \rho_\pi(s,a)(-\phi(\gamma)) - \rho_{\pi_E}(s,a)\phi(-\gamma) \quad (45)$$

$$= -R_\phi(\rho_\pi, \rho_{\pi_E}) \quad (46)$$

where we made the change of variables $c \rightarrow -\phi(\gamma)$, justified because T is the range of $-\phi$. $\square$

Having showed how to construct a cost function regularizer ψ_ϕ from ϕ , we obtain, as a corollary, a cost function regularizer for the logistic loss, whose optimal expected risk is, up to a constant, the Jensen-Shannon divergence.

Corollary A.1.1. *The cost regularizer (13)*

$$\psi_{GA}(c) \triangleq \begin{cases} \mathbb{E}_{\pi_E}[g(c(s,a))] & \text{if } c < 0 \\ +\infty & \text{otherwise} \end{cases} \quad \text{where } g(x) = \begin{cases} -x - \log(1 - e^x) & \text{if } x < 0 \\ +\infty & \text{otherwise} \end{cases}$$

satisfies

$$\psi_{GA}^*(\rho_\pi - \rho_{\pi_E}) = \max_{D \in (0,1)^{S \times \mathcal{A}}} \mathbb{E}_\pi[\log(D(s,a))] + \mathbb{E}_{\pi_E}[\log(1 - D(s,a))]. \quad (47)$$

Proof. Using the logistic loss $\phi(x) = \log(1 + e^{-x})$, we see that Eq. (40) reduces to the claimed ψ_{GA} . Applying Proposition A.1, we get

$$\psi_{GA}^*(\rho_\pi - \rho_{\pi_E}) = -R_\phi(\rho_\pi, \rho_{\pi_E}) \quad (48)$$

$$= \sum_{s,a} \max_{\gamma \in \mathbb{R}} \rho_\pi(s,a) \log\left(\frac{1}{1 + e^{-\gamma}}\right) + \rho_{\pi_E}(s,a) \log\left(\frac{1}{1 + e^\gamma}\right) \quad (49)$$

$$= \sum_{s,a} \max_{\gamma \in \mathbb{R}} \rho_\pi(s,a) \log\left(\frac{1}{1 + e^{-\gamma}}\right) + \rho_{\pi_E}(s,a) \log\left(1 - \frac{1}{1 + e^{-\gamma}}\right) \quad (50)$$

$$= \sum_{s,a} \max_{\gamma \in \mathbb{R}} \rho_\pi(s,a) \log(\sigma(\gamma)) + \rho_{\pi_E}(s,a) \log(1 - \sigma(\gamma)), \quad (51)$$

where $\sigma(x) = 1/(1 + e^{-x})$ is the sigmoid function. Because the range of σ is $(0, 1)$, we can write

$$\psi_{GA}^*(\rho_\pi - \rho_{\pi_E}) = \sum_{s,a} \max_{d \in (0,1)} \rho_\pi(s,a) \log d + \rho_{\pi_E}(s,a) \log(1 - d) \quad (52)$$

$$= \max_{D \in (0,1)^{S \times \mathcal{A}}} \sum_{s,a} \rho_\pi(s,a) \log(D(s,a)) + \rho_{\pi_E}(s,a) \log(1 - D(s,a)), \quad (53)$$

which is the desired expression. $\square$

We conclude with a policy gradient formula for causal entropy.

Lemma A.1. *The causal entropy gradient is given by*

$$\nabla_\theta \mathbb{E}_{\pi_\theta}[-\log \pi_\theta(a|s)] = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a|s) Q_{\log}(s, a)], \quad (54)$$

where $Q_{\log}(\bar{s}, \bar{a}) = \mathbb{E}_{\pi_\theta}[-\log \pi_\theta(a|s) | s_0 = \bar{s}, a_0 = \bar{a}]$.

Proof. For an occupancy measure $\rho(s, a)$, define $\rho(s) = \sum_a \rho(s, a)$. Next,

$$\begin{aligned}\nabla_{\theta} \mathbb{E}_{\pi_{\theta}}[-\log \pi_{\theta}(a|s)] &= -\nabla_{\theta} \sum_{s,a} \rho_{\pi_{\theta}}(s, a) \log \pi_{\theta}(a|s) \\ &= -\sum_{s,a} (\nabla_{\theta} \rho_{\pi_{\theta}}(s, a)) \log \pi_{\theta}(a|s) - \sum_s \rho_{\pi_{\theta}}(s) \sum_a \pi_{\theta}(a|s) \nabla_{\theta} \log \pi_{\theta}(a|s) \\ &= -\sum_{s,a} (\nabla_{\theta} \rho_{\pi_{\theta}}(s, a)) \log \pi_{\theta}(a|s) - \sum_s \rho_{\pi_{\theta}}(s) \sum_a \nabla_{\theta} \pi_{\theta}(a|s)\end{aligned}$$

The second term vanishes, because $\sum_a \nabla_{\theta} \pi_{\theta}(a|s) = \nabla_{\theta} \sum_a \pi_{\theta}(a|s) = \nabla_{\theta} 1 = 0$. We are left with

$$\nabla_{\theta} \mathbb{E}_{\pi_{\theta}}[-\log \pi_{\theta}(a|s)] = \sum_{s,a} (\nabla_{\theta} \rho_{\pi_{\theta}}(s, a)) (-\log \pi_{\theta}(a|s)),$$

which is the policy gradient for RL with the fixed cost function $c_{\log}(s, a) \triangleq -\log \pi_{\theta}(a|s)$. The resulting formula is given by the standard policy gradient formula for $c_{\log}$. $\square$

B Environments and detailed results

The environments we used for our experiments are from the OpenAI Gym [5]. The names and version numbers of these environments are listed in Table 1, which also lists dimension or cardinality of their observation and action spaces (numbers marked “continuous” indicate dimension for a continuous space, and numbers marked “discrete” indicate cardinality for a finite space).

Table 1: Environments

Task	Observation space	Action space	Random policy performance	Expert performance
Cartpole-v0	4 (continuous)	2 (discrete)	18.64 ± 7.45	200.00 ± 0.00
Acrobot-v0	4 (continuous)	3 (discrete)	-200.00 ± 0.00	-75.25 ± 10.94
Mountain Car-v0	2 (continuous)	3 (discrete)	-200.00 ± 0.00	-98.75 ± 8.71
Reacher-v1	11 (continuous)	2 (continuous)	-43.21 ± 4.32	-4.09 ± 1.70
HalfCheetah-v1	17 (continuous)	6 (continuous)	-282.43 ± 79.53	4463.46 ± 105.83
Hopper-v1	11 (continuous)	3 (continuous)	14.47 ± 7.96	3571.38 ± 184.20
Walker-v1	17 (continuous)	6 (continuous)	0.57 ± 4.59	6717.08 ± 845.62
Ant-v1	111 (continuous)	8 (continuous)	-69.68 ± 111.10	4228.37 ± 424.16
Humanoid-v1	376 (continuous)	17 (continuous)	122.87 ± 35.11	9575.40 ± 1750.80

The amount of environment interaction used for FEM, GTAL, and our algorithm is shown in Table 2. To reduce gradient variance for these three algorithms, we also fit value functions, with the same neural network architecture as the policies, and employed generalized advantage estimation [27] (with $\gamma = .995$ and $\lambda = .97$). The exact experimental results are listed in Table 3. Means and standard deviations are computed over 50 trajectories. For the cartpole, mountain car, acrobot, and reacher, these statistics are further computed over 7 policies learned from random initializations.

Table 2: Parameters for FEM, GTAL, and Algorithm 1

Task	Training iterations	State-action pairs per iteration
Cartpole	300	5000
Mountain Car	300	5000
Acrobot	300	5000
Reacher	200	50000
HalfCheetah	500	50000
Hopper	500	50000
Walker	500	50000
Ant	500	50000
Humanoid	1500	50000

Table 3: Learned policy performance

Task	Dataset size	Behavioral cloning	FEM	GTAL	Ours
Cartpole	1	72.02 $\pm$ 35.82	200.00 $\pm$ 0.00	200.00 $\pm$ 0.00	200.00 $\pm$ 0.00
	4	169.18 $\pm$ 59.81	200.00 $\pm$ 0.00	200.00 $\pm$ 0.00	200.00 $\pm$ 0.00
	7	188.60 $\pm$ 29.61	200.00 $\pm$ 0.00	199.94 $\pm$ 1.14	200.00 $\pm$ 0.00
	10	177.19 $\pm$ 52.83	199.75 $\pm$ 3.50	200.00 $\pm$ 0.00	200.00 $\pm$ 0.00
Acrobot	1	-130.60 $\pm$ 55.08	-133.14 $\pm$ 60.80	-81.35 $\pm$ 22.40	-77.26 $\pm$ 18.03
	4	-93.20 $\pm$ 32.58	-94.21 $\pm$ 47.20	-94.80 $\pm$ 46.08	-83.12 $\pm$ 23.31
	7	-96.92 $\pm$ 34.51	-95.08 $\pm$ 46.67	-95.75 $\pm$ 46.57	-82.56 $\pm$ 20.95
	10	-95.09 $\pm$ 33.33	-77.22 $\pm$ 18.51	-94.32 $\pm$ 46.51	-78.91 $\pm$ 15.76
Mountain Car	1	-136.76 $\pm$ 34.44	-100.97 $\pm$ 12.54	-115.48 $\pm$ 36.35	-101.55 $\pm$ 10.32
	4	-133.25 $\pm$ 29.97	-99.29 $\pm$ 8.33	-143.58 $\pm$ 50.08	-101.35 $\pm$ 10.63
	7	-127.34 $\pm$ 29.15	-100.65 $\pm$ 9.36	-128.96 $\pm$ 46.13	-99.90 $\pm$ 7.97
	10	-123.14 $\pm$ 28.26	-100.48 $\pm$ 8.14	-120.05 $\pm$ 36.66	-100.83 $\pm$ 11.40
HalfCheetah	4	-493.62 $\pm$ 246.58	734.01 $\pm$ 84.59	1008.14 $\pm$ 280.42	4515.70 $\pm$ 549.49
	11	637.57 $\pm$ 1708.10	-375.22 $\pm$ 291.13	226.06 $\pm$ 307.87	4280.65 $\pm$ 1119.93
	18	2705.01 $\pm$ 2273.00	343.58 $\pm$ 159.66	1084.26 $\pm$ 317.02	4749.43 $\pm$ 149.04
	25	3718.58 $\pm$ 1856.22	502.29 $\pm$ 375.78	869.55 $\pm$ 447.90	4840.07 $\pm$ 95.36
Hopper	4	50.57 $\pm$ 0.95	3571.98 $\pm$ 6.35	3065.21 $\pm$ 147.79	3614.22 $\pm$ 7.17
	11	1025.84 $\pm$ 266.86	3572.30 $\pm$ 12.03	3502.71 $\pm$ 14.54	3615.00 $\pm$ 4.32
	18	1949.09 $\pm$ 500.61	3230.68 $\pm$ 4.58	3201.05 $\pm$ 6.74	3600.70 $\pm$ 4.24
	25	3383.96 $\pm$ 657.61	3331.05 $\pm$ 3.55	3458.82 $\pm$ 5.40	3560.85 $\pm$ 3.09
Walker	4	32.18 $\pm$ 1.25	3648.17 $\pm$ 327.41	4945.90 $\pm$ 65.97	4877.98 $\pm$ 2848.37
	11	5946.81 $\pm$ 1733.73	4723.44 $\pm$ 117.18	6139.29 $\pm$ 91.48	6850.27 $\pm$ 39.19
	18	1263.82 $\pm$ 1347.74	4184.34 $\pm$ 485.54	5288.68 $\pm$ 37.29	6964.68 $\pm$ 46.30
	25	1599.36 $\pm$ 1456.59	4368.15 $\pm$ 267.17	4687.80 $\pm$ 186.22	6832.01 $\pm$ 254.64
Ant	4	1611.75 $\pm$ 359.54	-2052.51 $\pm$ 49.41	-5743.81 $\pm$ 723.48	3186.80 $\pm$ 903.57
	11	3065.59 $\pm$ 635.19	-4462.70 $\pm$ 53.84	-6252.19 $\pm$ 409.42	3306.67 $\pm$ 988.39
	18	2597.22 $\pm$ 1366.57	-5148.62 $\pm$ 37.80	-3067.07 $\pm$ 177.20	3033.87 $\pm$ 1460.96
	25	3235.73 $\pm$ 1186.38	-5122.12 $\pm$ 703.19	-3271.37 $\pm$ 226.66	4132.90 $\pm$ 878.67
Humanoid	80	1397.06 $\pm$ 1057.84	5093.12 $\pm$ 583.11	5096.43 $\pm$ 24.96	10200.73 $\pm$ 1324.47
	160	3655.14 $\pm$ 3714.28	5120.52 $\pm$ 17.07	5412.47 $\pm$ 19.53	10119.80 $\pm$ 1254.73
	240	5660.53 $\pm$ 3600.70	5192.34 $\pm$ 24.59	5145.94 $\pm$ 21.13	10361.94 $\pm$ 61.28
Task	Dataset size	Behavioral cloning	Ours ($\lambda = 0$)	Ours ($\lambda = 10^{-3}$)	Ours ($\lambda = 10^{-2}$)
Reacher	4	-10.97 $\pm$ 7.07	-67.23 $\pm$ 88.99	-32.37 $\pm$ 39.81	-46.72 $\pm$ 82.88
	11	-6.23 $\pm$ 3.29	-6.06 $\pm$ 5.36	-6.61 $\pm$ 5.11	-9.26 $\pm$ 21.88
	18	-4.76 $\pm$ 2.31	-8.25 $\pm$ 21.99	-5.66 $\pm$ 3.15	-5.04 $\pm$ 2.22